Горский Илья

Лекция: Typescript part 1

Горский Илья

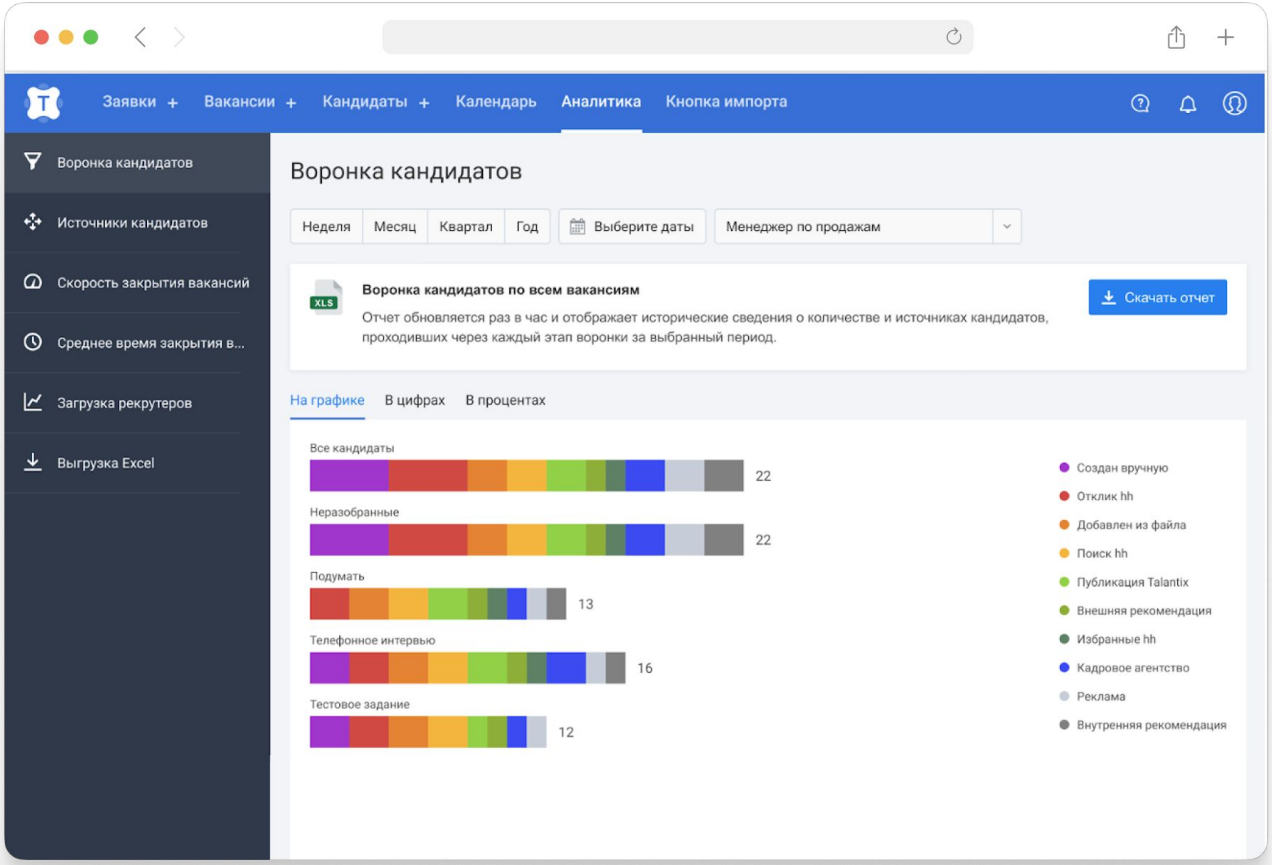
Руководитель разработки frontend talantix.ru

- 10 лет в разработке, последние 6 лет в hh.ru.
- Успел поработать в СМИ, стартапах и агентствах
- Ежегодно читаю лекции по TypeScript для студентов «Школы программистов hh.ru»
- Увлекаюсь архитектурой и проектированием дизайн-систем

Работаю в продукте Talantix

Talantix

Облачная CRM для ведения подбора (система для ведения процесса подбора) для клиентов среднего-крупного сегмента.



Часть 1: Типизация в языках программирования

Часть 2: Попытка типизировать web

Часть 3: Типизированный javascript

Часть I

Типизация

Типизация

1

Виды

Типизация

1

Виды

2

Характеристики

Типизация

1

Виды

2

Характеристики

3

Признаки

Типизация

1 Виды

2 Характеристики

3 Признаки

4 Совместимость типов

Виды

Динамическая типизация (Dynamic typing)

dynamic.js

```
let value;

value = 2021;
value = 'text'
```

Динамическая типизация (Dynamic typing)

+

Легкость в освоении.
Нет необходимости описания типов

–

Связывание типов происходит только в момент выполнения кода

Статическая типизация (Static typing)

static

```
string value;

value = 'test';
value = 2021; // Error
```

Слабая vs Строгая типизация

⚡ Слабая (JavaScript)

```
// Неявное приведение типов
"5" + 3          // "53"
"5" - 3          // 2
true + true      // 2
[] + {}          // "[object Object]"
null + 1         // 1
```

Компилятор сам решает, как привести типы.
Результат часто непредсказуем.

🛡️ Строгая (Java)

```
// Ошибка на этапе компиляции
int a = "5";     // ❌ Error!
int b = "5" - 3; // ❌ Error!

// Только явное приведение
int c = Integer.parseInt("5");
c + 3; // 8
```

Компилятор запрещает неявное приведение несовместимых типов.

Статическая типизация (Static typing)

+

Анализ кода
Рефакторинг
Уменьшение типовых ошибок

–

Порог входа
Скорость разработки

Виды типизации

1

Динамическая типизация (JS, Python, PHP)

👍 Легкость в освоении
👍 Нет необходимости описания типов

👎 Связывание типов происходит только в момент выполнения кода

2

Статическая типизация (C#, JAVA)

👍 Анализ кода
👍 Рефакторинг
👍 Уменьшение типовых ошибок

👎 Порог входа
👎 Скорость разработки

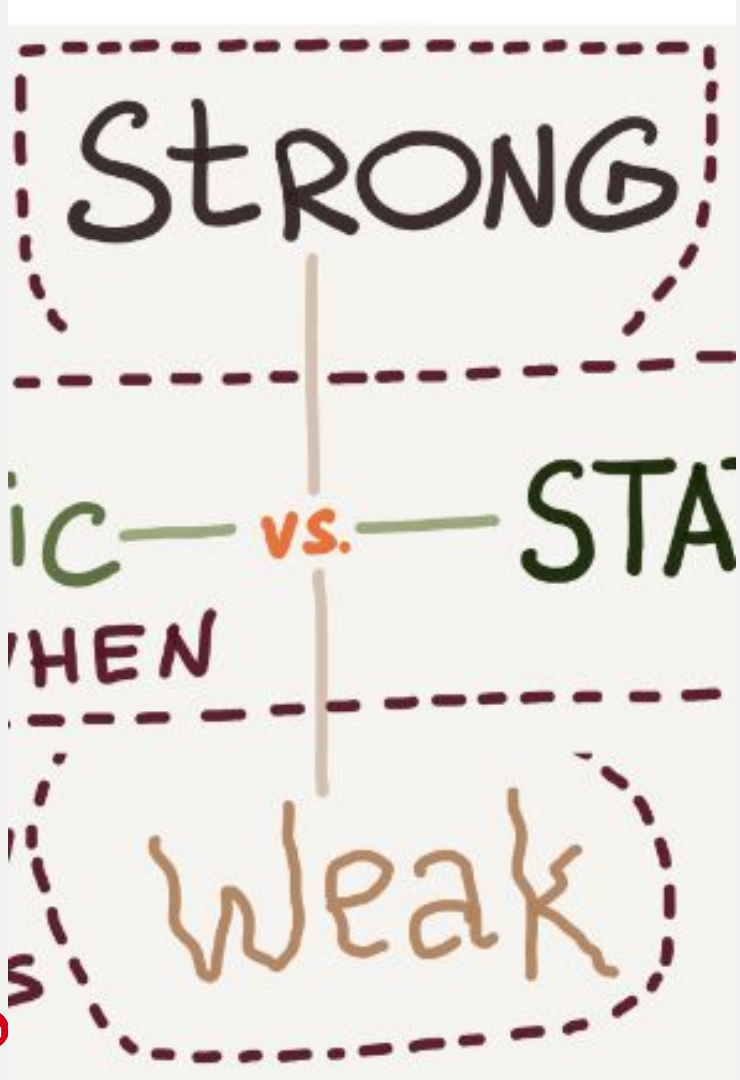
Типизация

1

Виды

2

Характеристики



Характеристики

Слабая типизация (Weak typing)

weak typing

```
var personsCount = 6;
var personsTitle = 'Всего '  + personsCount + ' персон';

console.log(personsTitle); // Всего 6 персон
```

Строгая типизация (Strong typing)

strong-typing.java

```
int personsCount = 5;
String personTitle = "Всего " + personsCount + 'персон'; // Error
```

Типизация

1

Виды

2

Характеристики

3

Признаки

Признаки

Неявная (Implicit typing)

Implicit typing.js

```
const name = 'Ilya';
const age = 1;
```

Явная (Explicit typing)

Explicit typing.java

```
String name = "Ilya";
int age = 28;
```

Типизация

1

Виды

2

Характеристики

3

Признаки

4

Совместимость типов

Совместимость типов

Номинативная типизация (Nominative typing)

Устанавливает совместимость типов, основываясь на ссылках

Структурная типизация (Structural typing)

Это принцип, определяющий совместимость типов, основываясь на их описании

Номинативная vs Структурная

Номинативная (Java)

```
class Cat { String name; }
class Dog { String name; }

Cat cat = new Cat();
Dog dog = new Dog();

cat = dog; // ❌ Error!
// Разные имена = разные типы
```

Совместимость по имени типа (наследование)

Структурная (TypeScript)

```
type Cat = { name: string }
type Dog = { name: string }

let cat: Cat = { name: "Tom" }
let dog: Dog = { name: "Rex" }

cat = dog; // ✅ OK!
// Одинаковая форма =
совместимы
```

Совместимость по форме (структуре)

Типизация

Типизация

- Динамическая / Статистическая

Типизация

⛓️ Динамическая / Статистическая

💪 Сильная / Слабая

Типизация

⛓️ Динамическая / Статистическая

💪 Сильная / Слабая

🧐 Явная / Неявная

Типизация

⛓️ Динамическая / Статистическая

💪 Сильная / Слабая

🧐 Явная / Неявная

🦆 Структурная / Номинативная

Часть II

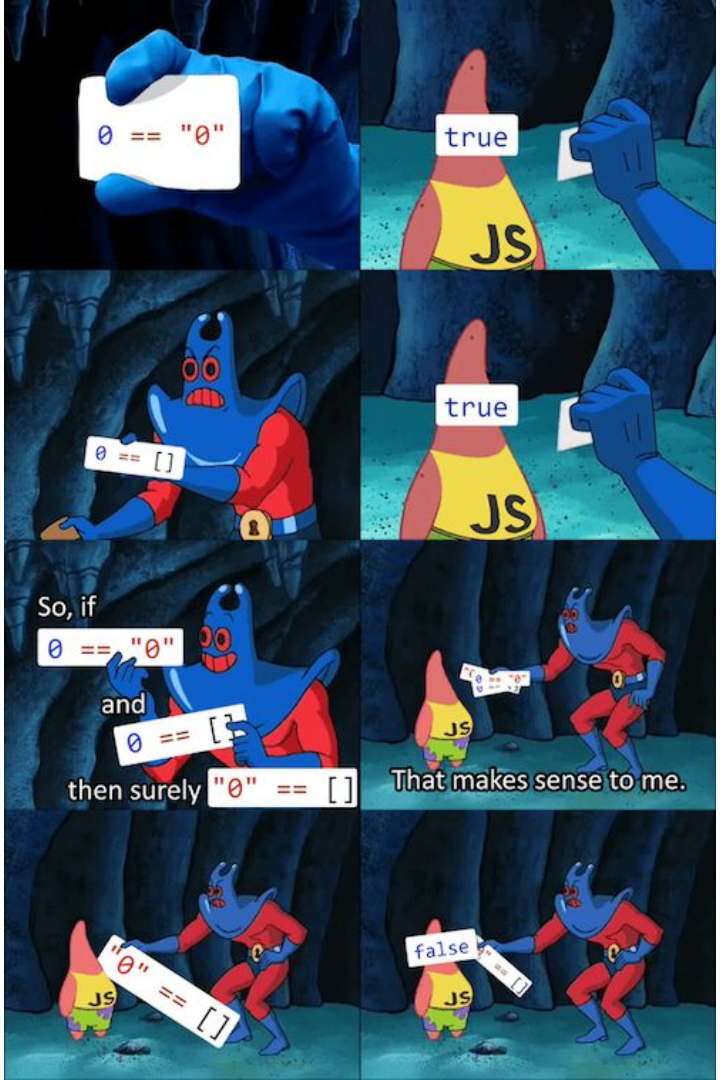
Неявное приведение типов

Неявное приведение типов

```
" " == [' '] // ?
```

Неявное приведение типов

```
[] == 0 // ?
```


0 == "0"
true
JS
0 == []
true
JS
So, if
0 == "0"
and
0 == []
then surely "0" == []
JS
That makes sense to me.
"0" == []
JS
false
JS

0 == "0"
true

"0" == []
false
[] == []

Динамическая типизация

```
> const SuperLib = { push () {} };
<· undefined
> SuperLib.pop()
✕ ▸Uncaught TypeError: SuperLib.pop is not a function
      at <anonymous>:1:10
>
```

Попытка типизировать веб

ECMAScript 4: версия, которой не было

4th Edition (abandoned) [edit]

The proposed fourth edition of ECMA-262 (**ECMAScript 4** or **ES4**) would have been the first major update to ECMAScript since the third edition was published in 1999. The specification (along with a reference implementation) was originally targeted for completion by October 2008.[18] The first draft was dated February 1999.[19] An overview of the language was released by the working group on 23 October 2007.[20]

By August 2008, the ECMAScript 4th edition proposal had been scaled back into a project code named ECMAScript Harmony. Features under discussion for Harmony at the time included:

- classes,
- a module system,
- optional type annotations and static typing, probably using a structural type system,
- generators and iterators,
- destructuring assignment, and
- algebraic data types.

The intent of these features was partly to better support *programming in the large*, and to allow sacrificing some of the script's ability to be dynamic to improve performance. For example, Tamarin – the virtual machine for ActionScript, developed and open-sourced by Adobe – has just-in-time compilation (JIT) support for certain classes of scripts.

Языки с собственным синтаксисом

Dart

elm

Языки, расширяющие синтаксис javascript

Языки, расширяющие синтаксис javascript

TypeScript: факты

Anders Hejlsberg

Open Source

Fixed 15% bugs*

typestudy.pdf (earlbarr.com)

TypeScript: почему именно он

Open Source

MIT License

Strict Superset

Любой .js = валидный .ts

-15% багов

typestudy.pdf (UCL)

⛓️ Статическая + Структурная + Явная/Неявная типизация

🔧 Автор: Anders Hejlsberg (создатель C# и Delphi)

🏢 Adoption: Angular, Vue 3, Svelte, Next.js, Deno

📦 Любой JS-код и npm-пакет работает из коробки

Часть III

Практика

Примитивы

number, string, bigint,
boolean, symbol

Ссылочные типы

object, array, tuple, enum

Специальные типы

undefined, null, any,
unknown, never, void

Type Aliases

union, intersection,
структурная совместимость

Literal Types

строковые, числовые,
template literal

Функции и интерфейсы

параметры, optional,
readonly, extends

🔴 Live Coding →

Спасибо